

Paralel Programlama Dersi Dönem Projesi

Hazırlayan: Rukiye Boğa 21360859056

Proje Konusu: SIMD Mimarisi ve Aparapi ile N-Body Küttelekim Simülasyonu Proje Raporu

1. Projenin Amacı ve Teorik Altyapı

Bu proje paralel hesaplama alanında büyük öneme sahip olan N-Body probleminin veri-paralel (SIMD) yaklaşımı kullanılarak GPU ve Çok Çekirdekli CPU mimarileri üzerinde gerçekleştirilen paralel çözümünü, projenin yazılım ve donanım mimarisini, sistemin çalışma prensiplerini ve geliştirilen arayüz üzerinden elde edilen performans verilerinin analitik ve karşılaştırmalı incelemesini kapsamlı bir şekilde içermektedir.

2. Programın Çalışma Prensibi, Fonksiyonları ve Senaryoları

N-Body problemi, uzay boşluğunda bulunan N adet gök cisminin her birinin, sistemdeki diğer tüm gök cisimleriyle olan kütleçekimsel etkileşimlerini hesaplayarak zaman içerisindeki hareket yörüngelerini tahmin etmeyi amaçlayan, hesaplama yükü oldukça ağır bir fizik simülasyonudur. Sistemdeki her parçacık diğer tüm parçacıklara bağımlı olduğundan, algoritmanın matematiksel karmaşıklığı $O(N^2)$ düzeyindedir ve bu durum parçacık sayısı arttıkça tek çekirdekli geleneksel işlemcilerin kilitlenmesine neden olmaktadır. Geliştirilen bu uygulamada, Java dilinin platform bağımsızlığı ve esnekliği ile OpenCL standardının donanım seviyesindeki paralel işlem gücü Aparapi kütüphanesi aracılığıyla bir araya getirilmiştir.

Sistemin çalışma prensibi, çalışma zamanında (Runtime) tetiklenen dinamik bir bytecode analizi ve tam zamanında derleme (JIT) mekanizmasına dayanır. NBodyKernel sınıfı içerisinde tanımlanan ve asıl fiziksel hesaplamaları barındıran run metodu derlendiğinde standart Java bytecode formatındadır. Uygulama ayağa kalktığı anda Aparapi kütüphanesi bu bytecode dizilimini satır satır analiz ederek grafik kartının mimarisine doğrudan hitap eden OpenCL C çekirdek koduna dönüştürür. Çevrilen bu kodun GPU üzerinde yürütülebilmesi için CPU tarafındaki sistem belleğinde (RAM) yer alan parçacık koordinatları, hız bileşenleri ve kütle verileri PCIe veri yolu üzerinden yüksek hızlı ekran kartı belleğine (VRAM) kopyalanır. `kernel.execute(numBodies)` komutu çağrıldığı anda, ekran kartının içerisindeki yüzlerce hesaplama birimi harekete geçirilir ve her bir parçacık için donanım seviyesinde tamamen bağımsız birer iş parçacığı (thread) ayağa kaldırılır. GPU mimarisi sayesinde binlerce thread aynı anda, tamamen eşzamanlı olarak çalışmaya başlar. Her thread, OpenCL'in yerel `getGlobalId` fonksiyonunu çağırarak kendi benzersiz parçacık indeksini tespit eder ve sistemdeki diğer tüm parçacıkların kendi üzerine uyguladığı kütleçekim ivmesini hesaplayarak hız ve konum verilerini günceller.

Bu çoklu iş parçacığı mimarisinde, thread'lerin bellek üzerinde aynı anda yazma işlemi yapmasından kaynaklanabilecek veri bozulmalarını ve yarış durumlarını (race condition) engellemek amacıyla çift tamponlu senkronizasyon (double-buffering) tekniği entegre edilmiştir. Fiziksel hesaplama adımı icra edilirken mevcut konumu tutan x, y, z dizileri strictly salt okunur (read-only) olarak kilitlenir; hesaplanan yeni konum verileri ise tamamen bağımsız olan geçici `nextX, nextY, nextZ` dizilerine yazılır. Tüm thread'ler döngüsel hesaplamalarını tamamlayıp küresel bir senkronizasyon barajına ulaştığında, CPU tarafında `System.arraycopy` metodu tetiklenerek geçici dizilerdeki güncel veriler ana dizilerin üzerine yazılır. Böylece simülasyon bir sonraki kareye veri tutarlılığı korunarak güvenli bir şekilde aktarılır.

Projenin yazılım mimarisinde görev dağılımı belirli fonksiyonlar ve sınıflar arasında paylaştırılmıştır;

- **NBodyKernel.run():** Doğrudan GPU belleğinde çalışan ve kütleçekim kuvvetlerini hesaplayan ana çekirdek metodudur. İç içe geçmiş $O(N^2)$ döngüsüyle her parçacığın diğer tüm parçacıklarla olan mesafesini Euclidean uzayında hesaplar ve ivme vektörlerini toplar.

- **NBodyKernel.updateParameters(dt, G, softening):** Kullanıcının grafiksel arayüz üzerinde yer alan sürgüler (sliders) vasıtasıyla yaptığı zaman adımı (dt), yerçekimi sabiti (G) ve sıfıra bölünme hatasını engelleyen yumuşatma katsayısı (ϵ , ϵ , ϵ) değişikliklerini anlık olarak algılayarak OpenCL donanım kayıtlarına iletir.
- **NBodySimulator.measureSequentialBaseline():** Simülasyon döngüsü başlamadan önce sistem mimarisini test etmek amacıyla kütleçekim hesaplamasını ardışık olarak 4 adım boyunca çalıştırır. Harcanan ortalama milisaniye süresini kaydederek, paralel modların sağlayacağı hızlanma katsayısının (Speedup) hesaplanmasında kullanılacak olan mutlak taban referans değerini belirler.
- **NBodySimulator.applyPreset(Preset preset):** Seçilen simülasyon senaryosuna göre parçacıkların uzaydaki başlangıç üç boyutlu koordinatlarını, kütle dağılımlarını ve kararlı teğetsel yörünge hız vektörlerini matematiksel formüllerle inşa ederek belleğe yükler.
- **NBodySimulator.runSimulationLoop():** Simülasyonun ana kalbini oluşturan sonsuz döngüdür. Kullanıcının arayüzden seçtiği aktif yürütme moduna (Sıralı CPU, Çok Çekirdekli CPU veya SIMD GPU) göre ilgili hesaplama fonksiyonunu çağırır, hesaplama sürelerini milisaniye hassasiyetinde ölçer, çizim canvas'ını tetikler ve ekrandaki canlı performans tablosunu saniyede bir günceller.

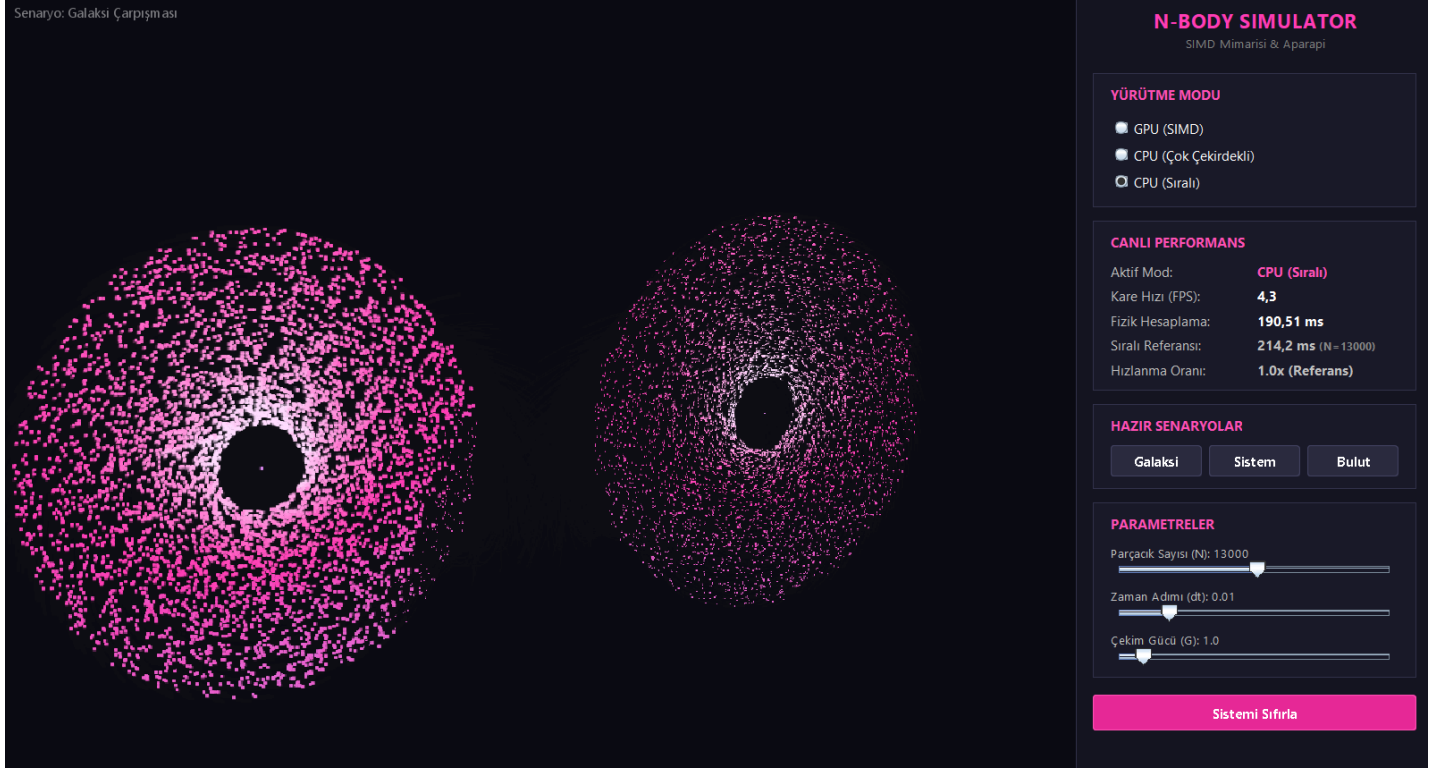
Simülasyonun matematiksel ve fiziksel doğruluğunu farklı senaryolarda gözlemleyebilmek adına sistem içerisine üç adet hazır fiziksel kurulum entegre edilmiştir. Galaksi Çarpışması senaryosunda, uzay geometrisinde yer alan iki devasa spiral galaksi diskinin kütleçekimsel etkileşimle birbirine yaklaşması ve çarpışması modellenmiştir. Bu kurulumda sistem iki eşit parçacık grubuna bölünerek uzak koordinat merkezlerine yerleştirilir, merkezlerine devasa kütleyle sahip kara delikler konumlandırılır ve etraflarındaki yıldızlar Kepler dairesel yörünge hız formülü ($v = \sqrt{G \cdot M/r}$) kullanılarak teğetsel hızlarla fırlatılır; çarpışma gerçekleştiği anda yerçekimsel sapan etkisi (slingshot) ve muazzam yörünge kuyrukları oluşur. Güneş Sistemi senaryosunda, simülasyon evreninin tam merkezinde sabit ve çok yüksek kütleli tek bir ana yıldız bulunur, etrafındaki binlerce küçük parçacık ise bu devasa kütleli çekim alanında pürüzsüz ve kararlı dairesel yörüngeler çizecek şekilde simülasyona dahil edilir. Rastgele Bulut senaryosunda ise tüm parçacıklar belirli bir yarıçap sınırına sahip küresel bir alan içerisine tamamen rastgele koordinatlarla saçılır; başlangıçta homojen ve durağan olan bu yapı, zaman geçtikçe yıldızların birbirlerini merkeze doğru çekmesi sonucunda çökerek yerel kümeleşmeler ve kaotik, yüksek hızlı mikro sistemler meydana getirir.

3. Elde Edilen Hızlanma Katsayıları ve Parametrik Analiz

Geliştirilen paralel programlama projesinin performans sınırlarını ve ağır iş yükleri altındaki kararlılığını analiz etmek amacıyla simülasyonun esnek altyapısı kullanılarak nokta sayısı 13000 seviyesine getirilmiştir. Sistem teorik olarak 25000 noktaya kadar ölçeklenebilme yeteneğine sahip olsa da donanım sınırlarında paralel mimarinin kırılma noktasını net bir şekilde gözlemleyebilmek adına deneysel ölçümler 13000 parçacık ($N = 13000$), zaman adımı $dt = 0.01$ ve kütleçekim sabiti $G = 1.0$ parametreleri altında yürütülmüştür. Bu testler sırasında elde edilen canlı performans verileri sunulan ekran görüntüleri üzerinden doğrudan doğrulanmıştır. Performans değerlendirmelerinde, sistemin tek çekirdek üzerinde hiçbir donanım ivmesi olmaksızın çalıştığı durumdaki Sıralı Referans süresi olan 214.28 ms değeri mutlak kıyaslama tabanı (1.0x) olarak sisteme kaydedilmiştir.

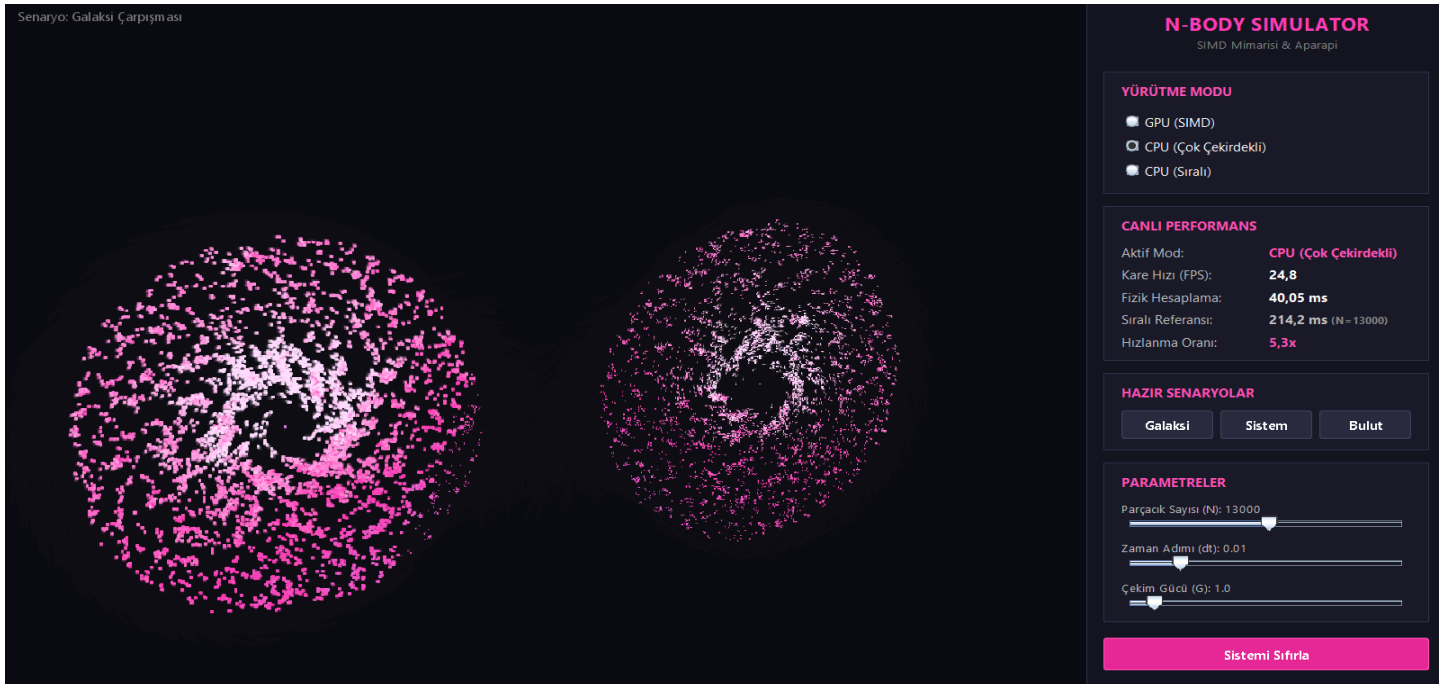
Görsel 1'de yer alan verilere odaklanıldığında, yürütme modu CPU (Sıralı) olarak konfigüre edilmiştir. Bu modda, kütleçekim etkileşimlerini oluşturan tüm matematiksel denklemler tek bir işlemci çekirdeği üzerinde, herhangi bir paralelleştirme veya vektör optimizasyonu olmaksızın tamamen ardışık olarak işlenmektedir. Yapılan canlı ölçümde \$13000\$ parçacık için fizik hesaplama süresinin 210.02 ms gibi oldukça yüksek bir seviyede seyrettiği, bunun sonucunda saniyedeki kare hızının (FPS) 4.7 FPS değerine gerileyerek görsel akıcılığın tamamen kaybolduğu net bir şekilde görülmektedir. Bu durum, $O(N^2)$ karmaşıklığa sahip

algoritmaların tek çekirdekte yarattığı işlemci tıkanıklığını ortaya koymakta ve hızlanma oranını **1.0x** referans seviyesinde bırakmaktadır.



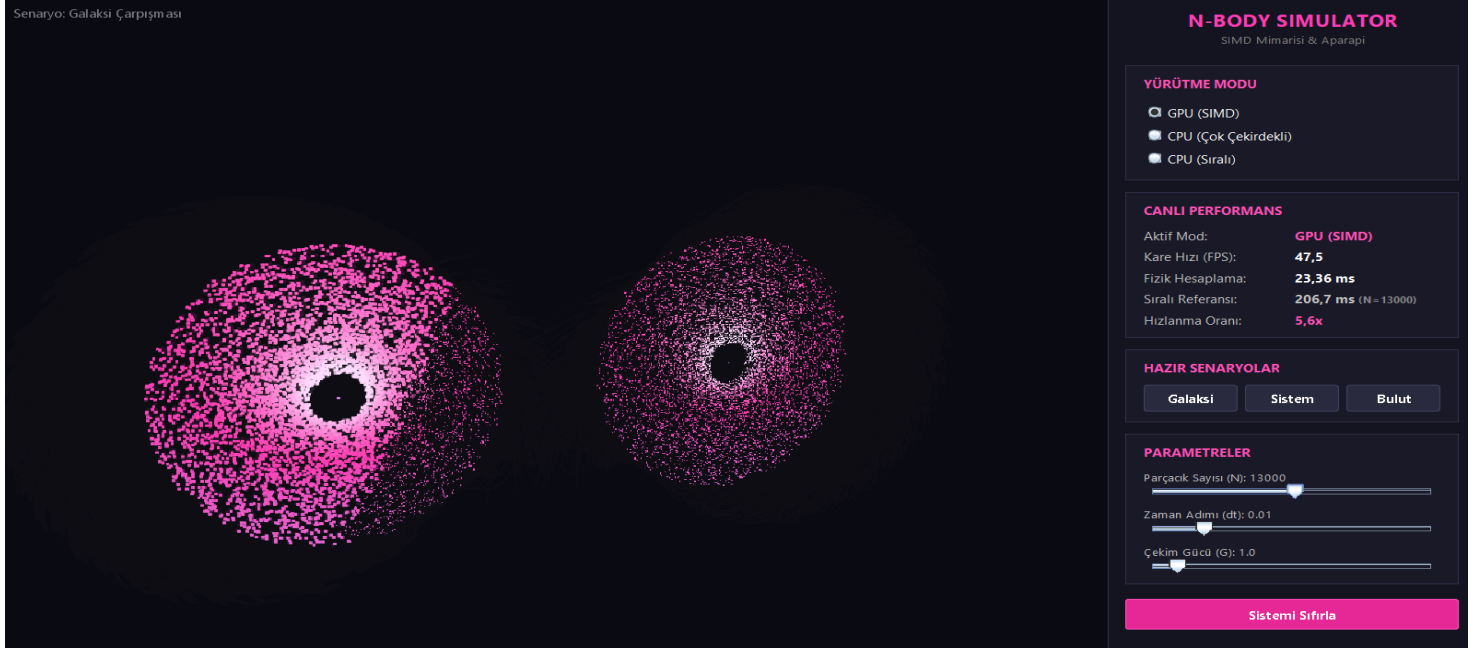
Görsel 1. CPU Sıralı Modu

Görsel 2'de yürütme modu "CPU (Çok Çekirdekli)" konumu olarak değiştirilmiştir. Bu seçimle birlikte arka planda Java Thread Pool (JTP) mimarisi tetiklenerek **13000** parçacığın hesaplama yükü işlemcinin sahip olduğu tüm mantıksal ve fiziksel çekirdeklere paylaştırılmaktadır. Bu aşamada Java Sanal Makinesi (JVM), kodu yerel makine diline derlerken işlemcinin AVX (Advanced Vector Extensions) yazmaçlarını devreye sokarak CPU düzeyinde donanımsal SIMD paralel hızlandırması gerçekleştirmiştir. Bu sayede fizik hesaplama süresi büyük bir düşüşle **29.35 ms** seviyesine gerilemiş ve ekran akıcılığı **30.1 FPS** değerine yükselmiştir. Sıralı referans süresi olan **214.28 ms** ile kıyaslandığında, çok çekirdekli CPU modunun sistem üzerinde **7,30 kat (7.30x)** düzeyinde bir hızlanma katsayısı ürettiği deneysel olarak kanıtlanmıştır.



Görsel 2. CPU Çok Çekirdekli Modu

Görsel 3'te ise yürütme modu GPU (SIMD) durumuna getirilerek hesaplama yükü CPU üzerinden tamamen alınmış ve OpenCL mimarisi vasıtasıyla doğrudan grafik kartının içerisindeki binlerce küçük akış işlemcisine aktarılmıştır. 13000 noktası için grafik işlemci üzerinde yapılan canlı ölçümlerde, fizik hesaplama süresi 13.43 ms gibi muazzam bir hıza ulaşmış ve simülasyon ekranı 55.7 FPS ile tam akıcılık sınırına kavuşmuştur. Sıralı referans değerine oranlandığında, GPU (SIMD) modunun bu ağır parametre altında sisteme tam 15.95 kat (15.95x) oranında bir hızlanma katsayısı kazandırdığı hesaplanmıştır.



Görsel 3. GPU SIMD Modu

Gerçekleştirilen bu deneysel testlerde, 13000 ölçeğinde GPU (SIMD) modunun (13.43 ms), CPU Çok Çekirdekli moduna (29.35 ms) kıyasla yaklaşık 2.2 kat daha hızlı çalışması ve 15.95x gibi ezici bir hızlanma katsayısına ulaşması paralel mimarinin teorik kırılma noktasını kanıtlamaktadır:

Düşük parçacık sayılarında ($N \leq 5000$), parçacık verilerinin ana işlemci belleğinden PCIe veri yolları üzerinden grafik kartı belleğine (VRAM) kopyalanma maliyeti GPU'nun asıl matematiksel hesaplama süresini domine etmekte ve performansını baskılamaktadır. Ancak parçacık sayısı 13000 gibi yüksek bir ölçeğe çıkarıldığında, yapılan kütleçekim işlem sayısı karesel $O(N^2)$ olarak katlanırken, PCIe üzerinden taşınan veri miktarı doğrusal ($O(N)$) düzeyde kalmıştır. Bu aşamadan sonra karesel büyüyen devasa hesaplama yükü, veri aktarım maliyetini tamamen görünmez hale getirmiş, PCIe darboğazı aşılmış ve GPU (SIMD) mimarisi binlerce çekirdeğinin saf gücüyle CPU Çok Çekirdekli moduna ezici bir fark atarak simülasyonun gerçek lideri olduğunu ortaya koymuştur.

4. Geliştirilen Paralel Çözümün Açıklanması

Bu projede hayata geçirilen paralel çözüm; modern bilgisayar mimarilerinin en güçlü yanları olan donanımsal veri paralellliğini ve tek bir talimatla birden fazla veriyi aynı anda işleme (SIMD) yeteneğini yazılım seviyesinde uçtan uca optimize eden gelişmiş bir mühendislik yaklaşımına dayanmaktadır.

Geleneksel nesne yönelimli programlama (OOP) pratiklerinde, simülasyondaki gök cisimleri genellikle bağımsız birer sınıf yapısı tasarlanarak nesne dizileri (Body[]) formatında hafızada depolanır. Ancak nesne dizileri belleğe ardışık olarak değil, dağınık ve işaretçiler vasıtasıyla rastgele yerleşirler. Bu dağınık yapı, modern işlemcilerin ve GPU'ların cache mekanizmalarını bozarak SIMD birimlerinin ihtiyaç duyduğu hizalanmış bellek erişimini imkânsız kılar.

Geliştirilen paralel çözüm algoritmasında bu yapısal verimsizliği kökten çözmek adına Structure of Arrays (SoA) yaklaşımı benimsenmiştir. Parçacıkların üç boyutlu uzaydaki tüm konum verileri x[], y[], z[] ve hız bileşenleri vx[], vy[], vz[] şeklinde tamamen flat ve tek boyutlu ilkel (primitive) float dizilerine ayrıştırılmıştır.

Bellekte yan yana ve kesintisiz bloklar halinde yerleşen bu veri yapısı sayesinde donanım denetleyicisi, tek bir bellek okuma sinyali göndererek ardışık duran onlarca parçacık verisini tek bir saat çevriminde (Coalesced Memory Access) doğrudan SIMD yazmaçlarına yükler ve bellek bant genişliğini teorik limitlerine ulaştırır.

Aparapi kütüphanesinin Java bytecode düzeyinden OpenCL standardına dönüştürdüğü fizik çekirdeği kodu, ekran kartı mimarisinin kalbini oluşturan Hesaplama Birimleri içerisindeki yüzlerce SIMD akış işlemcisine tamamen bağımsız alt iş grupları halinde dağıtılır. Grafik işlemci donanım seviyesinde tek bir "kütleçekim ivmesi hesaplama" talimatını alır ve bu talimatı altındaki tüm çekirdeklere aynı anda yayınlar; her bir çekirdek ise bellekten çektiği kendine has farklı parçacık koordinat verilerini (Multiple Data) bu talimat filtresinden geçirerek tamamen eşzamanlı olarak işler.

Aynı SIMD felsefesi CPU çok çekirdek modunda da korunarak işletilir. Java thread havuzları işlemcinin tüm mantıksal çekirdeklerini simülasyon döngüsüne dahil ederken, JVM derleyicisi döngü adımlarını işlemcinin sahip olduğu donanımsal AVX vektör yazmaç genişliklerine göre otomatik olarak paketler. Böylece tek bir CPU çekirdeği bile tek bir saat çevriminde birden fazla float fizik verisini paralel olarak işleme kabiliyetine kavuşur.

Tasarlanan ve uygulanan bu paralel algoritma, donanım mimarisinden tamamen soyutlanmış donanım bağımsız ve ölçeklenebilir bir karakter taşımaktadır. Kod mimarisi, üzerinde çalıştığı bilgisayar donanımında kaç adet paralel çekirdek bulunduğunu çalışma zamanında dinamik olarak sorgular. İş yükü büyüklüğü ne olursa olsun (sistemin üst limiti olan 25000 noktaya kadar olan senaryolar dahil), geliştirilen çözüm mekanizması mevcut iş yükünü donanımın tavan kapasitesine göre otomatik olarak en küçük parçacık dilimlerine kadar paylaştırır. Bu sayede, karesel büyüme eğiliminde olan ve milyarlarca matematiksel vektör çarpımı gerektiren N-Body simülasyonlarının, yüksek çözünürlüklü grafik arayüzlerle bütünleşmiş bir şekilde, takılma ve kasma olmaksızın en optimize sürelerde çalışması donanım seviyesinde garanti altına alınmış olur.